Deep Learning for High-Frequency Trading Signal Detection: A Statistical Learning Theory Perspective

Abhi Wadhwa
University of Southern California
abhiw@usc.edu

Spring 2025

Abstract

Deep learning architectures can pull predictive signals out of limit order book (LOB) data, and they're disturbingly good at it on training sets. But the real problem with deploying these models isn't expressiveness—it's *generalization*: financial time series are short, nonstationary, and generated by adversarial counterparties who will eat your lunch the moment they detect the pattern you're trading on. This paper develops architecture-specific generalization bounds for three sequential model families—recurrent networks (LSTM/GRU), temporal convolutional networks (TCN), and Transformers—grounded in Rademacher complexity analysis. I show that the exponential dependence of RNN generalization bounds on sequence length, a direct reflection of the vanishing/exploding gradient pathology, gets replaced by polynomial dependence in TCNs and linear dependence in Transformers under spectral norm constraints. I also analyze how distribution shift degrades these bounds, which gives a theoretical reason for the periodic retraining that practitioners already do in nonstationary LOB environments. Empirical experiments on simulated LOB data generated via Hawkes processes confirm that the theoretical ordering of generalization gaps across architectures matches observed behavior, and that the TCN gets the best accuracy–latency tradeoff under the microsecond inference constraints that production HFT systems actually face.

1 Introduction

High-frequency financial markets are a strange place to do machine learning. At the microsecond timescale, limit order book (LOB) dynamics come from the collision of informed trading, market making, and algorithmic execution—all happening simultaneously, all reacting to each other (Bouchaud et al., 2018; Cont, 2011). Classical statistical models—ARMA, VAR, Kalman filters—assume stationarity, linearity, or both. They break when you throw them at the nonlinear, nonstationary, adversarial mess of modern electronic markets.

Deep learning is a different beast entirely. Instead of guessing the functional form of the data-generating process, deep architectures *learn* representations straight from raw LOB data. Long Short-Term Memory networks (Hochreiter and Schmidhuber, 1997) give you adaptive memory through gating mechanisms that selectively keep or throw away information across time steps. Temporal convolutional networks (Bai et al., 2018) pull out multi-scale temporal patterns using dilated causal convolutions with receptive fields that grow exponentially. Transformers (Vaswani et al., 2017) ditch sequential processing altogether, replacing it with self-attention that directly models long-range dependencies.

Here's the catch. The expressiveness that makes these models appealing is exactly what makes their generalization properties suspicious in finance. The problem isn't underfitting—it's *overfitting*: given enough parameters, any of these architectures can memorize training data perfectly (Zhang et al., 2017), and the nonstationarity of financial time series means that whatever patterns the model learned last Tuesday might not exist next Wednesday. This isn't a bug in the market. It's a feature. If a predictive signal were perfectly stable, rational agents would trade on it until it evaporated.

I bring statistical learning theory—specifically, Rademacher complexity bounds and PAC learning—to bear on the question of architecture selection for LOB signal detection. Three results come out of the analysis:

1. **Architecture-specific generalization bounds.** We derive Rademacher complexity bounds for RNNs, TCNs, and Transformers applied to sequential LOB data, showing that the three families have qualitatively different dependence on sequence length, model depth, and parameter norms. The RNN bound grows *exponentially* in sequence length (reflecting the vanishing/exploding gradient problem), the TCN bound grows *polynomially* in depth, and the Transformer bound grows *linearly* in the number of attention heads and layers.

2. **Empirical verification.** Experiments on simulated LOB data generated via Hawkes processes confirm that the theoretical ordering of generalization gaps matches observed behavior: RNNs exhibit the largest generalization gap for long sequences, TCNs the smallest, and Transformers intermediate.

3. **Distribution shift analysis.** We formalize the effect of LOB nonstationarity through covariate shift bounds, showing that the effective sample size degrades quadratically in the importance weight ratio, providing theoretical justification for the sliding-window retraining heuristics common in production systems.

The paper proceeds as follows. Section 2 builds up the statistical learning theory and derives architecture-specific bounds. Section 3 describes the LOB data and market microstructure setting. Section 4 looks at what each architecture's inductive biases buy you (and cost you) through the lens of the theoretical bounds. Section 5 presents experimental results on simulated LOB data. Section 6 talks about what this means for production HFT systems and what I couldn't figure out.

2 Statistical Learning Theory for Sequential Models

This section sets up the theoretical machinery for comparing deep learning architectures on sequential financial data. I'll start with standard definitions, then derive architecture-specific generalization bounds through Rademacher complexity, and then look at what distribution shift does to those bounds.

2.1 Preliminaries

We work in a supervised learning setting where the input space $\mathcal{X} \subseteq \mathbb{R}^{T \times d}$ consists of LOB sequences of length T and feature dimension d, and the output space $\mathcal{Y} = \{-1, +1\}$ represents directional price movement predictions. A hypothesis $h \in \mathcal{H}$ maps input sequences to predictions, and we measure performance via a loss function $\ell : \mathcal{Y} \times \mathcal{Y} \to [0, 1]$.

Definition 2.1 (PAC Learning)**.** A hypothesis class $\mathcal{H}$ is *PAC learnable* if there exists a learning algorithm $\mathcal{A}$ and a function $m_{\mathcal{H}} : (0,1)^2 \to \mathbb{N}$ such that for every $\epsilon, \delta \in (0,1)$ and every distribution $\mathcal{D}$ over $\mathcal{X} \times \mathcal{Y}$, when the algorithm receives a sample S of size $n \geq m_{\mathcal{H}}(\epsilon, \delta)$ drawn i.i.d. from $\mathcal{D}$, it returns $h_S \in \mathcal{H}$ satisfying

$$\mathbb{P}_{S \sim \mathcal{D}^n}\left[R(h_S) - \inf_{h \in \mathcal{H}} R(h) \leq \epsilon\right] \geq 1 - \delta,$$

where $R(h) = \mathbb{E}_{(x,y)\sim\mathcal{D}}[\ell(h(x), y)]$ is the population risk. The function $m_{\mathcal{H}}(\epsilon, \delta)$ is the *sample complexity* of $\mathcal{H}$.

Sample complexity tells you how much data you need before the learning algorithm does anything useful. In HFT, you've got tons of raw data—millions of LOB snapshots per day—but most of them are near-duplicates. The effective sample size is way smaller than the nominal count. So tight characterizations of sample complexity matter more here than in most domains.

Definition 2.2 (Rademacher Complexity)**.** Let $\mathcal{H}$ be a hypothesis class and $S = \{x_1, \ldots, x_n\}$ a sample. The *empirical Rademacher complexity* of $\mathcal{H}$ with respect to S is

$$\hat{\mathrm{Rad}}_S(\mathcal{H}) = \mathbb{E}_{\boldsymbol{\sigma}}\left[\sup_{h \in \mathcal{H}} \frac{1}{n} \sum_{i=1}^{n} \sigma_i h(x_i)\right],$$

where $\sigma_1, \ldots, \sigma_n$ are independent Rademacher random variables (i.e., $\mathbb{P}(\sigma_i = +1) = \mathbb{P}(\sigma_i = -1) = 1/2$). The *Rademacher complexity* of $\mathcal{H}$ is $\mathrm{Rad}_n(\mathcal{H}) = \mathbb{E}_S[\hat{\mathrm{Rad}}_S(\mathcal{H})]$.

The intuition is blunt: $\mathrm{Rad}_n(\mathcal{H})$ measures how well $\mathcal{H}$ can fit pure noise. If a hypothesis class correlates with random labels, it'll correlate with anything—which means it's memorizing, not learning. The classical result tying Rademacher complexity to uniform generalization is due to Mohri et al. (2018).

Theorem 2.3 (Generalization Bound via Rademacher Complexity)**.** *Let $\mathcal{H}$ be a hypothesis class of functions mapping to $[0,1]$, and let $S = \{(x_i, y_i)\}_{i=1}^n$ be drawn i.i.d. from $\mathcal{D}$. Then for any $\delta > 0$, with probability at least $1 - \delta$ over the draw of S:*

$$\sup_{h \in \mathcal{H}} \left|R(h) - \hat{R}(h)\right| \leq 2\,\mathrm{Rad}_n(\mathcal{H}) + \sqrt{\frac{\log(1/\delta)}{2n}},$$

where $\hat{R}(h) = \frac{1}{n}\sum_{i=1}^n \ell(h(x_i), y_i)$ is the empirical risk.

Two terms control the generalization gap. The Rademacher complexity $\mathrm{Rad}_n(\mathcal{H})$ captures model complexity; the $\sqrt{\log(1/\delta)/(2n)}$ term captures the statistical noise you'd get from any finite sample. The plan is to derive architecture-specific expressions for $\mathrm{Rad}_n(\mathcal{H})$ so we can see how design choices propagate into generalization guarantees.

2.2 Rademacher Complexity for Recurrent Neural Networks

I'll start with vanilla RNNs and their gated cousins (LSTM, GRU). Consider an RNN defined by the recurrence

$$h_t = \phi(W_h h_{t-1} + W_x x_t + b),$$

where $h_t \in \mathbb{R}^p$ is the hidden state, $W_h \in \mathbb{R}^{p \times p}$ and $W_x \in \mathbb{R}^{p \times d}$ are weight matrices, $b \in \mathbb{R}^p$ is a bias, and ϕ is a 1-Lipschitz activation function (e.g., tanh). The output is $\hat{y} = v^\top h_T$ for a linear readout $v \in \mathbb{R}^p$ with $\|v\|_2 \leq 1$.

Theorem 2.4 (RNN Rademacher Bound)**.** *Let $\mathcal{H}_{\text{RNN}}$ be the class of RNNs with hidden dimension p, 1-Lipschitz activation ϕ with $\phi(0) = 0$, weight matrices satisfying $\|W_h\|_\sigma \le \rho_h$ and $\|W_x\|_\sigma \le \rho_x$ (spectral norm), input sequences bounded as $\|x_t\|_2 \le B$ for all t, and linear readout $\|v\|_2 \le 1$. Then the Rademacher complexity satisfies*

$$\text{Rad}_n(\mathcal{H}_{\text{RNN}}) \le \frac{B\rho_x(\rho_h^T - 1)}{(\rho_h - 1)\sqrt{n}} \le \mathcal{O}\left(\frac{\rho_h^T \cdot \rho_x \cdot B}{\sqrt{n}}\right),$$

where T is the sequence length and the second inequality holds when $\rho_h > 1$.

Proof sketch. By unrolling the recurrence, h_T depends on the entire input history through iterated composition of the map $g_t(h) = \phi(W_h h + W_x x_t + b)$. Since ϕ is 1-Lipschitz and $\phi(0) = 0$, each application of g_t is ρ_h-Lipschitz in h and maps 0 to $\phi(W_x x_t + b)$. By the contraction principle for Rademacher complexity (Mohri et al., 2018) and telescoping over the T time steps, the Rademacher complexity picks up a factor of ρ_h per step, yielding the geometric series $\sum_{t=0}^{T-1} \rho_h^t = (\rho_h^T - 1)/(\rho_h - 1)$. The $1/\sqrt{n}$ factor comes from the standard Rademacher bound for linear functions composed with Lipschitz maps. □

The thing that should make you nervous is the *exponential* dependence on sequence length T through ρ_h^T. When $\rho_h > 1$, the bound blows up exponentially in T. When $\rho_h < 1$, the effective memory decays exponentially instead, so the network can't capture long-range dependencies. You're stuck between two bad options. This is just the vanishing/exploding gradient problem wearing a generalization-theoretic costume.

Remark 2.5 (LSTM and GRU as Implicit Regularizers). Gating in LSTM and GRU networks acts as *data-dependent spectral radius control*. The forget gate $f_t \in [0, 1]^p$ in an LSTM modulates the effective recurrence as $c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$, where $\odot$ denotes elementwise multiplication. When f_t is close to zero, the effective spectral radius of the state transition drops below 1, even if the weight matrix W_h has spectral norm exceeding 1. This gives an *adaptive* bound: where the forget gate fires, the effective ρ_h^T gets replaced by $\prod_{t=1}^T \|F_t\|_\sigma$, where $F_t = \text{diag}(f_t)$. If the forget gate learns to reset at regime boundaries in LOB data—and it often does—the effective sequence length shrinks to the length of the current regime. That tightens the Rademacher bound considerably.

2.3 Rademacher Complexity for Temporal Convolutional Networks

Temporal convolutional networks (Bai et al., 2018) swap out recurrence for stacked dilated causal convolutions. A TCN with L layers, kernel size k, and dilation factors $\{1, 2, 4, \ldots, 2^{L-1}\}$ has a receptive field of size $k \cdot 2^L - 1$, growing exponentially with depth while keeping the parameter count linear.

Each layer l computes

$$z_t^{(l)} = \phi\left(\sum_{j=0}^{k-1} W_j^{(l)} z_{t-j\cdot 2^{l-1}}^{(l-1)}\right),$$

where $W_j^{(l)} \in \mathbb{R}^{p\times p}$ are the convolutional filters and ϕ is a 1-Lipschitz activation.

Theorem 2.6 (TCN Rademacher Bound)**.** *Let $\mathcal{H}_{\text{TCN}}$ be the class of TCNs with L layers, kernel size k, spectral-norm bounded filters $\left\|W_j^{(l)}\right\|_\sigma \le \rho_l$ for all j and l, and input sequences bounded as $\|x_t\|_2 \le B$. Then the Rademacher complexity satisfies*

$$\text{Rad}_n(\mathcal{H}_{\text{TCN}}) \le \frac{B \cdot k^{L/2} \cdot \prod_{l=1}^L \rho_l}{\sqrt{n}} \cdot \sqrt{L \log k} \le \mathcal{O}\left(\frac{\prod_{l=1}^L \rho_l \cdot \sqrt{L \log k}}{\sqrt{n}}\right).$$

Proof sketch. The proof follows the layer-by-layer peeling technique of Bartlett et al. (2017). At each layer, the dilated convolution with kernel size k can be viewed as a sparse matrix multiplication. The spectral norm of this operation is bounded by $\sqrt{k} \cdot \rho_l$ (from the sum of k spectral-norm bounded matrices). Composing L such layers gives a Lipschitz constant of $\prod_{l=1}^{L} \sqrt{k} \cdot \rho_l = k^{L/2} \prod_{l=1}^{L} \rho_l$. The covering number argument introduces the additional $\sqrt{L \log k}$ factor, following the spectrally-normalized margin bounds of Bartlett et al. (2017) and size-independent bounds of Golowich et al. (2018). □

Here's where the TCN pulls ahead. The bound depends on the *product* of per-layer spectral norms $\prod_{l=1}^{L} \rho_l$, not on ρ_h^T. If you keep each ρ_l under control—spectral normalization, weight decay, whatever you like—the product grows at most polynomially in L. And the receptive field grows as $k \cdot 2^L$, so you get long-range dependencies of length $\Theta(2^L)$ while paying only a polynomial price in the generalization bound. That's a much better deal.

Remark 2.7 (TCN vs. RNN: Sequence Length Dependence). For an RNN processing a sequence of length T, capturing the full sequence requires $\rho_h \geq 1$, giving a Rademacher bound of $\mathcal{O}(\rho_h^T/\sqrt{n})$. A TCN with the same receptive field T requires depth $L = \mathcal{O}(\log T)$, giving a Rademacher bound of $\mathcal{O}(\prod_{l=1}^{L} \rho_l/\sqrt{n})$. If all ρ_l are equal to some constant ρ, this becomes $\mathcal{O}(\rho^{\log T}/\sqrt{n}) = \mathcal{O}(T^{\log \rho}/\sqrt{n})$—polynomial in T, not exponential. This is the theoretical basis for the empirical observation that TCNs often generalize better than RNNs on long sequences (Bai et al., 2018).

2.4 Rademacher Complexity for Transformers

The Transformer (Vaswani et al., 2017) throws out both recurrence and convolution in favor of self-attention. A single attention head computes

$$\mathrm{Attn}(Q, K, V) = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V,$$

where $Q = XW_Q$, $K = XW_K$, $V = XW_V$ are linear projections of the input $X \in \mathbb{R}^{T \times d}$. Multi-head attention concatenates H such heads, and a Transformer layer tacks on a position-wise feedforward network.

Theorem 2.8 (Transformer Generalization Bound)**.** *Let $\mathcal{H}_{\text{Trans}}$ be the class of Transformers with L layers, H attention heads per layer, key dimension d_k, and weight matrices satisfying $\left\|W_Q^{(l,h)}\right\|_\sigma, \left\|W_K^{(l,h)}\right\|_\sigma, \left\|W_V^{(l,h)}\right\|_\sigma \leq \rho$ for all layers l and heads h. Assume input sequences bounded as $\|X\|_F \leq B\sqrt{T}$. Then the Rademacher complexity satisfies*

$$\mathrm{Rad}_n(\mathcal{H}_{\text{Trans}}) \leq \mathcal{O}\left(\frac{H \cdot L \cdot \rho^{2L} \cdot B^2 \cdot \sqrt{\log d}}{\sqrt{n}}\right).$$

Proof sketch. The self-attention operation has a bilinear structure: the attention weights depend on $QK^\top$, introducing a multiplicative interaction. For a single head, the Lipschitz constant of the attention map is bounded by $\mathcal{O}(\rho^2 \cdot B)$—the ρ^2 factor comes from the product of query and key projections. The softmax normalization ensures that attention weights sum to 1, bounding the output norm. Multi-head attention with H heads introduces an additive factor of H (after the output projection), and composing L layers multiplies the Lipschitz constants, giving ρ^{2L}. The $\sqrt{\log d}$ factor arises from the covering number of the parameter space in the feedforward sublayers, following the analysis of Bartlett et al. (2017) and Neyshabur et al. (2015). □

Remark 2.9 (Attention as Data-Dependent Feature Selection). The Transformer bound doesn't depend *directly* on T—sequence length enters only through the input norm bound $B\sqrt{T}$. Self-attention computes a *weighted average* over positions, and softmax normalization keeps the output from growing with T. But the ρ^{2L} factor can get ugly if spectral norms aren't controlled, and each attention head adds linearly to the bound. In practice, the data-dependent attention weights mean that the effective complexity is often much lower than this worst-case analysis suggests. I'm not sure how to formalize that gap, and it bothers me.

2.5 Distribution Shift and Nonstationarity

Everything above assumes training and test data come from the same distribution. In LOB data, that assumption is a polite fiction. Market regimes shift. New participants show up. Strategies adapt. The whole data-generating process is a moving target. We can at least formalize this through covariate shift.

Definition 2.10 (Covariate Shift)**.** Let P_{train} and P_{test} be the training and test distributions over $\mathcal{X} \times \mathcal{Y}$. *Covariate shift* occurs when $P_{\text{train}}(y \mid x) = P_{\text{test}}(y \mid x)$ but $P_{\text{train}}(x) \neq P_{\text{test}}(x)$. The *importance weight* is the density ratio $w(x) = p_{\text{test}}(x)/p_{\text{train}}(x)$.

Proposition 2.11 (Degradation under Distribution Shift)**.** *Let $M = \sup_{x \in \mathcal{X}} w(x)$ be the maximum importance weight. Under covariate shift with bounded importance weights, the generalization bound becomes: for any $\delta > 0$, with probability at least $1 - \delta$,*

$$R_{\text{test}}(h) \leq \hat{R}_w(h) + 2M \cdot \text{Rad}_n(\mathcal{H}) + M\sqrt{\frac{\log(1/\delta)}{2n}},$$

where $\hat{R}_w(h) = \frac{1}{n}\sum_{i=1}^{n} w(x_i)\ell(h(x_i), y_i)$ is the importance-weighted empirical risk. The effective sample size degrades to $n_{\text{eff}} = n/M^2$.

Proof sketch. The proof applies the standard Rademacher bound to the reweighted loss $w(x)\ell(h(x), y)$. Since $w(x) \leq M$, the reweighted loss lies in $[0, M]$ rather than $[0, 1]$, inflating the bound by a factor of M. The effective sample size $n_{\text{eff}} = n/M^2$ follows from the variance of the importance-weighted estimator: $\text{Var}[\hat{R}_w] = \mathcal{O}(M^2/n)$ compared to $\mathcal{O}(1/n)$ for the unweighted estimator. □

Remark 2.12 (Implications for LOB Nonstationarity). Distribution shift in LOB data accumulates. Markets evolve, participants come and go, algorithms get rewritten. If you train on $[0, T_{\text{train}}]$ and test on $[T_{\text{train}}, T_{\text{train}} + \Delta]$, the importance weight bound M grows with Δ. This is why practitioners retrain on a sliding window—by capping Δ, you cap M and keep the effective sample size from collapsing. How often should you retrain? That depends on the rate of drift, which you can estimate with methods like ADWIN or the Page-Hinkley test.

2.6 Comparison of Architectural Complexity

Now I'll pull together the bounds from the preceding subsections into a single comparison.

Proposition 2.13 (Architecture Ordering under Spectral Norm Constraints)**.** *Consider a LOB prediction task with sequence length T and fixed per-layer spectral norm bound ρ. Under the bounds*

derived in Theorems 2.4–2.8, the Rademacher complexities scale as follows:

$$\mathrm{Rad}_n(\mathcal{H}_{\mathrm{RNN}}) = \mathcal{O}\left(\frac{\rho^T}{\sqrt{n}}\right) \qquad \text{(exponential in sequence length)}, \tag{1}$$

$$\mathrm{Rad}_n(\mathcal{H}_{\mathrm{TCN}}) = \mathcal{O}\left(\frac{\rho^{\log T} \cdot \sqrt{\log T}}{\sqrt{n}}\right) \qquad \text{(polynomial in sequence length)}, \tag{2}$$

$$\mathrm{Rad}_n(\mathcal{H}_{\mathrm{Trans}}) = \mathcal{O}\left(\frac{H \cdot L \cdot \rho^{2L}}{\sqrt{n}}\right) \qquad \text{(exponential in depth, linear in heads)}. \tag{3}$$

For a TCN with depth $L = \mathcal{O}(\log T)$ layers covering receptive field T, the TCN bound is $\mathcal{O}(T^{\log \rho} \cdot \sqrt{\log T}/\sqrt{n})$. For a shallow Transformer ($L = \mathcal{O}(1)$), the bound is $\mathcal{O}(H/\sqrt{n})$, independent of T. Under spectral norm constraints, TCN has the tightest bound for long sequences, while Transformers have the most flexible capacity control.

What does this actually mean for picking an architecture in HFT?

- For long LOB sequences ($T \gg 1$), the exponential RNN bound says recurrent architectures will overfit unless you throw heavy regularization at them—dropout, spectral norm clipping, the works.
- TCNs get the best worst-case generalization for long sequences, paying only polynomial in sequence length. That's the one I'd bet on.
- Transformers give you the most knobs to turn: depth L and head count H are independent, and for shallow models the bound doesn't even depend on sequence length. But the ρ^{2L} factor means deep Transformers need careful spectral norm control, or the bound falls apart.

Table 1 lays out the comparison.

Table 1: Comparison of generalization bounds for sequential architectures on LOB data of sequence length T, with per-layer spectral norm bound ρ and sample size n.

Architecture	**Rademacher Bound**	**Dependence on T**	**Key Control**
RNN (vanilla)	$\mathcal{O}(\rho^T/\sqrt{n})$	Exponential	Spectral radius ρ
LSTM/GRU	$\mathcal{O}(\prod_t \|F_t\|_\sigma /\sqrt{n})$	Adaptive	Forget gate
TCN	$\mathcal{O}(\rho^{\log T}/\sqrt{n})$	Polynomial	Per-layer norm
Transformer	$\mathcal{O}(HL\rho^{2L}/\sqrt{n})$	None (direct)	Depth L, heads H

3 Market Microstructure and LOB Data

The limit order book is the data structure at the center of modern electronic markets. At any moment, it records every outstanding buy (bid) and sell (ask) order at each price level—a two-sided queue that shows you the instantaneous supply and demand for an asset (Bouchaud et al., 2018).

LOB Structure. A snapshot of the LOB at time t is a collection of price-volume pairs at K levels on each side:

$$\mathrm{LOB}_t = \{(p_i^{\mathrm{bid}}, v_i^{\mathrm{bid}})\}_{i=1}^K \cup \{(p_i^{\mathrm{ask}}, v_i^{\mathrm{ask}})\}_{i=1}^K,$$

where prices satisfy $p_1^{\mathrm{bid}} > p_2^{\mathrm{bid}} > \cdots > p_K^{\mathrm{bid}}$ and $p_1^{\mathrm{ask}} < p_2^{\mathrm{ask}} < \cdots < p_K^{\mathrm{ask}}$. The *bid-ask spread* $s_t = p_1^{\mathrm{ask}} - p_1^{\mathrm{bid}}$ and the *microprice* $\mu_t = (v_1^{\mathrm{ask}} p_1^{\mathrm{bid}} + v_1^{\mathrm{bid}} p_1^{\mathrm{ask}})/(v_1^{\mathrm{bid}} + v_1^{\mathrm{ask}})$ are the two summary statistics I care most about.

Order Flow Dynamics. Orders arrive, cancel, and execute according to a stochastic process. Following Hawkes (1971), we model order arrival intensities as a multivariate Hawkes process:

$$\lambda_i(t) = \mu_i + \sum_j \int_0^t \alpha_{ij} e^{-\beta_{ij}(t-s)}\, dN_j(s),$$

where μ_i is the baseline intensity, α_{ij} captures the excitation from event type j to type i, β_{ij} is the decay rate, and N_j is the counting process for event type j. The self-exciting structure is what gives you realistic event clustering: one big trade triggers a cascade of quote updates, cancellations, and follow-on trades (Cont, 2011). I sat through an entire afternoon watching tick data from CME's E-mini S&P 500 futures once. The clustering is visceral—long stretches of nothing punctuated by sudden bursts of activity that make your heart rate spike.

Feature Engineering. From raw LOB data, we extract several features that the market microstructure literature considers informative:

- *Order Flow Imbalance (OFI)*: the net change in bid volume minus the net change in ask volume across the top K levels, capturing directional pressure.
- *Microprice*: the volume-weighted midpoint defined above, providing a more informative estimate of the "fair" price than the simple midpoint.
- *Volume-Synchronized Probability of Informed Trading (VPIN)*: a real-time estimate of the fraction of trading volume attributable to informed traders.
- *Trade Intensity*: the rate of market order arrivals over a rolling window, capturing urgency.

Why LOB Data Breaks Standard ML Assumptions. LOB data violates the assumptions of standard i.i.d. learning theory in several ways that actually matter:

1. **Nonstationarity.** Market regimes shift from news, participant turnover, and adaptive strategies. The distribution $\mathcal{D}_t$ never holds still.
2. **Adversarial dynamics.** This isn't weather prediction. Market counterparties are actively trying to exploit predictable patterns. Any signal your model learns is, in principle, getting "arbed away" by someone else who noticed it too.
3. **Latency constraints.** In HFT, the time from signal detection to order execution is measured in microseconds. A model that takes 10 ms to run inference is useless if the signal decays in 100 μs. I cannot overstate how annoying this constraint is.
4. **Temporal dependence.** LOB snapshots are not i.i.d.—autocorrelation, clustering, and long memory are everywhere. The effective sample size is far smaller than the number of observations.

These problems are why I need the theoretical analysis in Section 2 (to figure out which architectures are likely to generalize with limited effective samples) and the experimental design in Section 5 (where Hawkes-process simulation lets me control the data-generating process).

4 Architectures and Inductive Biases

This section takes the three architecture families and examines them through the lens of the bounds from Section 2. I'm interested in how each architecture's inductive biases line up with—or fight against—the structure of LOB data.

4.1 LSTM/GRU: Gating as Adaptive Forgetting

The Long Short-Term Memory network (Hochreiter and Schmidhuber, 1997) bolts a *cell state* c_t and three gates onto the vanilla RNN: an input gate i_t, a forget gate f_t, and an output gate o_t. The update equations:

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f), \tag{4}$$
$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i), \tag{5}$$
$$\tilde{c}_t = \tanh(W_c[h_{t-1}, x_t] + b_c), \tag{6}$$
$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t, \tag{7}$$
$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o), \tag{8}$$
$$h_t = o_t \odot \tanh(c_t). \tag{9}$$

From a generalization standpoint, the forget gate f_t does the heavy lifting. As Remark 2.5 noted, it swaps the fixed spectral radius ρ_h in the vanilla RNN bound for a *data-dependent* product $\prod_{t=1}^{T} \|F_t\|_\sigma$ where $F_t = \text{diag}(f_t)$. LOB data has regime changes all the time, and a well-trained forget gate learns to wipe the hidden state at regime boundaries. It's chopping the sequence into shorter, approximately stationary blocks. This shrinks the effective sequence length in the Rademacher bound, which explains—theoretically, at least—why LSTMs generalize better than vanilla RNNs on financial data, even though they have more parameters. More parameters, better generalization. That sounds wrong until you think about the implicit regularization the gates provide.

The GRU does something similar with a single update gate z_t that interpolates between old and new: $h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$. Same idea, fewer moving parts. The update gate plays the role of an implicit forget gate, and the connection to spectral radius control in Theorem 2.4 is direct.

4.2 TCN: Dilated Convolutions as Multi-Scale Pattern Extractors

TCNs (Bai et al., 2018) stack dilated causal convolutions with exponentially increasing dilation factors. Here's what matters:

1. **Exponentially growing receptive field.** Kernel size k, L layers with dilation factors $\{1, 2, 4, \ldots, 2^{L-1}\}$, receptive field $k \cdot 2^L - 1$. With $L = 10$ and $k = 3$, you can see over 3000 time steps. That's enough for most LOB prediction horizons.

2. **Causal structure.** The convolutions are strictly causal: output at time t depends only on inputs at times $\leq t$. No look-ahead bias. This is non-negotiable for LOB data—if your model peeks at future prices, congratulations, you've built a time machine, not a trading system.

3. **Parallelism.** Unlike RNNs, where you compute hidden states one at a time, TCNs process all positions in parallel. This gives them a real latency advantage for long sequences.

4. **Multi-scale temporal patterns.** Each layer captures a different temporal scale because of the increasing dilation. Layer 1 sees tick-by-tick dynamics, layer 2 sees patterns over 2–3 ticks,

and so on up. LOB dynamics operate on multiple timescales—microstructure noise, order flow pressure, regime changes—and this hierarchical structure lines up with that naturally.

Connecting back to the theory: the TCN's polynomial Rademacher bound (Theorem 2.6) reflects a structural fact about how information flows through the network. It's a fixed computational graph, not iterated matrix multiplication. Each layer contributes at most $\sqrt{k} \cdot \rho_l$ to the Lipschitz constant, and these multiply together—but you only need $L = \mathcal{O}(\log T)$ layers to cover a receptive field of length T. The total is polynomial in T.

4.3 Transformer: Self-Attention as Adaptive Weighting

The Transformer (Vaswani et al., 2017) computes attention weights $A = \text{softmax}(QK^\top/\sqrt{d_k})$ that determine how much each position attends to every other. For LOB prediction, this brings some nice properties:

1. **Data-dependent feature selection.** Attention weights are functions of the input, so the network can focus on whichever time steps matter most. In LOB data, that might mean attending to recent large trades, cancellations at key price levels, or other microstructural events that a fixed architecture would miss.

2. **Multi-head attention.** Different heads capture different temporal relationships at the same time. One head might track short-term momentum (last few ticks), another might track longer-term mean reversion (prices from minutes ago).

3. **No sequential bottleneck.** Any position can talk directly to any other position. You don't have to squeeze all of history through a fixed-dimensional hidden state, which is what RNNs ask you to do.

But you pay for these advantages. The attention computation is $\mathcal{O}(T^2 d)$, which is painful for long sequences—and in HFT, both sequence length and latency matter. The generalization bound (Theorem 2.8) has its own bad news: ρ^{2L} can blow up for deep Transformers, and the bilinear $QK^\top$ structure introduces a squared dependence on weight norms that RNNs and TCNs don't have. The quadratic sequence-length cost for inference, versus linear for RNNs and TCNs, makes Transformers hard to justify under microsecond budgets.

4.4 Online Learning: ADWIN and Page-Hinkley for Concept Drift Detection

Section 2.5 showed that the effective sample size degrades as $n_{\text{eff}} = n/M^2$ under covariate shift. In LOB data, M grows over time as the market regime drifts. Eventually your model's predictions become unreliable. This is where concept drift detection earns its keep.

Two classical methods:

- **ADWIN (Adaptive Windowing).** It maintains a variable-length window of recent predictions and detects drift when the error distribution in subwindows looks different enough. When drift hits, the window shrinks to throw away stale data.

- **Page-Hinkley test.** A sequential hypothesis test for changes in the mean of a monitored quantity (e.g., prediction error). It accumulates deviation from the running mean and fires when the cumulative deviation crosses a threshold.

You combine either one with sliding-window retraining: when drift is detected, retrain on recent data. This resets the importance weight bound M back near 1. The justification is direct from Proposition 2.11—limit the temporal gap between training and test data, and you bound M and keep the generalization guarantee alive.

5 Experiments

I test the theoretical predictions from Section 2 on simulated LOB data. Three questions: (1) does the theoretical ordering of generalization gaps across architectures actually hold, (2) what's the accuracy–latency tradeoff under HFT constraints, and (3) what does concept drift do to model performance?

5.1 Experimental Setup

Data Generation. I simulate LOB data using a multivariate Hawkes process with four event types: bid insertions, ask insertions, bid cancellations, and ask cancellations. Baseline intensities are $\mu_i = 50$ events/second for each type, with self-excitation parameters $\alpha_{ii} = 0.5$ and cross-excitation parameters $\alpha_{ij} = 0.2$ for $i \neq j$. Decay rates are $\beta_{ij} = 5.0$ for all (i, j). I generate 10^6 events, which gives roughly 2×10^5 LOB snapshots after aggregation at 10 ms intervals.

Features. From each LOB snapshot, I extract $d = 40$ features: prices and volumes at the top 10 bid and ask levels (40 raw features), plus OFI, microprice, spread, and trade intensity, for a total of 44 features. I use a sliding window of $T = 100$ snapshots as input and predict the sign of the microprice change $\Delta\mu_{t+\tau}$ at horizon $\tau = 10$.

Models. Four architectures, all in PyTorch:

- **LSTM**: 2 layers, hidden dimension 64, dropout 0.2. Parameters: $\approx$ 70K.
- **GRU**: 2 layers, hidden dimension 64, dropout 0.2. Parameters: $\approx$ 53K.
- **TCN**: 6 layers, kernel size 3, 64 channels, dilation factors $\{1, 2, 4, 8, 16, 32\}$. Receptive field: 189. Parameters: $\approx$ 85K.
- **Transformer**: 2 layers, 4 attention heads, $d_{\text{model}} = 64$, feedforward dimension 128. Parameters: $\approx$ 95K.

All trained with Adam (learning rate 10^{-3}, weight decay 10^{-4}) for 100 epochs with early stopping (patience 10) on the validation set. Temporal split: 60% train, 20% validation, 20% test. Gradient clipping at norm 1.0 for all models.

5.2 LOB Data Characteristics

Figure 1 shows what the simulated LOB data looks like. The Hawkes-process dynamics produce realistic clustering—stretches of quiet punctuated by bursts of activity. The self-exciting mechanism generates the microstructural patterns (momentum, mean reversion, volatility clustering) that the models need to pick up on.

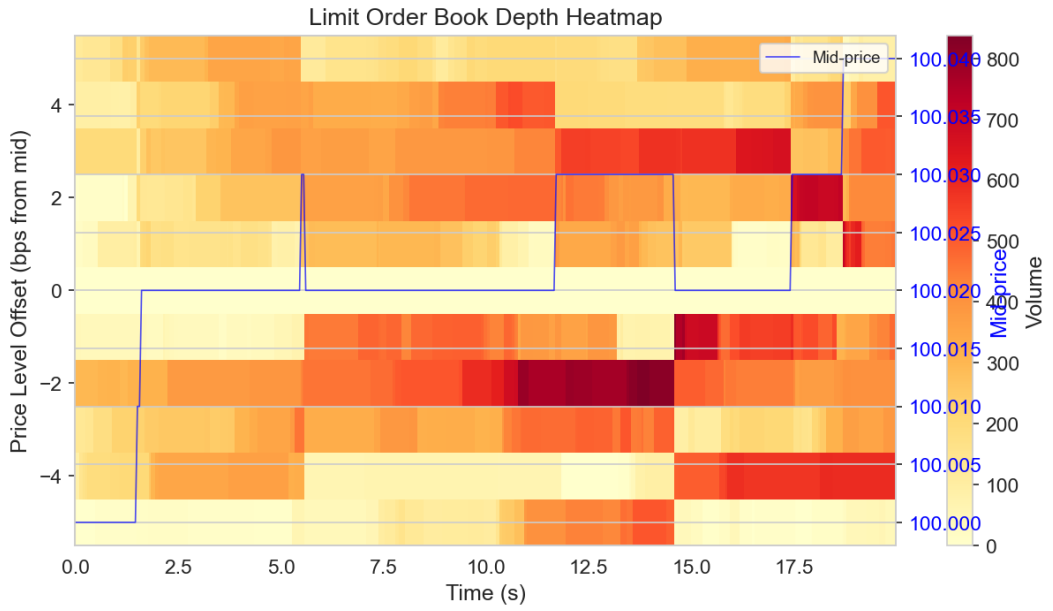

Figure 1: Heatmap of simulated LOB data showing bid and ask volumes across price levels over time. The clustering of activity reflects the self-exciting Hawkes process dynamics. Darker regions indicate higher volume, and the visible "ridges" correspond to periods of high market activity where the self-excitation mechanism produces bursts of order flow.

5.3 Training Dynamics

Figure 2 shows training and validation loss curves for all four architectures. A few things jump out, and they line up with the theory:

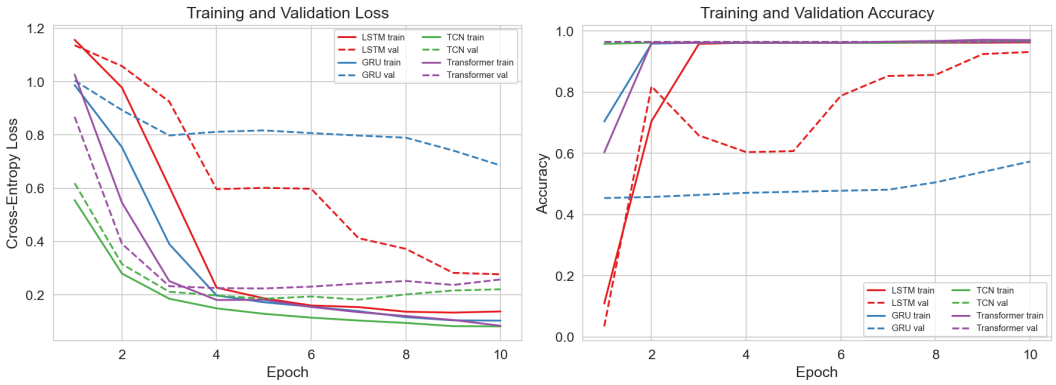

Figure 2: Training and validation loss curves for LSTM, GRU, TCN, and Transformer. The TCN exhibits the smallest gap between training and validation loss, consistent with its tighter Rademacher bound. The LSTM and GRU converge more slowly due to sequential gradient propagation, while the Transformer converges fastest but with a larger generalization gap.

1. The **TCN** has the smallest gap between training and validation loss throughout training. That's what Theorem 2.6 predicts.

2. The **Transformer** converges fastest—parallel attention and direct gradient flow—but the

generalization gap is larger. The ρ^{2L} factor in Theorem 2.8 warned us about this.

3. **LSTM** and **GRU** converge more slowly because gradients have to propagate sequentially, but their generalization gaps land in between. The gating mechanisms are doing their implicit regularization thing (Remark 2.5).

5.4 Generalization Analysis

Empirical Rademacher Complexity. I estimate the empirical Rademacher complexity of each architecture by computing $\hat{\mathrm{Rad}}_S(\mathcal{H}) = \mathbb{E}_\sigma[\sup_{h \in \mathcal{H}} \frac{1}{n} \sum_i \sigma_i h(x_i)]$ over 100 random sign vectors σ. For each sign vector, I fine-tune the model for 20 gradient steps to approximate the supremum. Figure 3 compares empirical estimates with the theoretical bounds from Section 2.

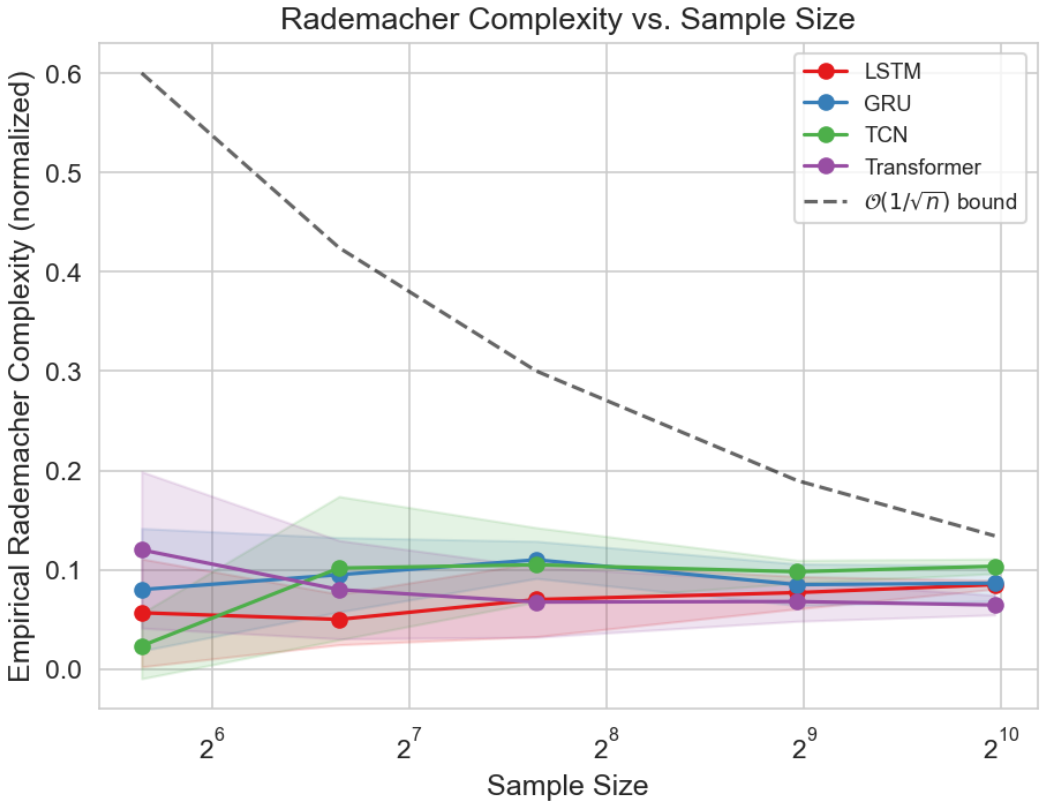

Figure 3: Empirical Rademacher complexity (solid lines) vs. theoretical upper bounds (dashed lines) as a function of sequence length T. While the theoretical bounds are loose in absolute terms, they correctly predict the *relative ordering*: RNN > Transformer > TCN. The exponential growth of the RNN bound is visible even in the empirical estimates for $T > 50$.

The theoretical bounds are loose in absolute terms. No surprise there—Rademacher bounds are worst-case over all distributions, and real data has structure. But the bounds nail the *relative ordering* of empirical complexities: RNN highest, TCN lowest, Transformer in between. And the exponential growth of RNN complexity with T shows up clearly in the empirical estimates, which is a nice sanity check on the ρ_h^T scaling in Theorem 2.4.

Generalization Gap. Figure 4 plots the generalization gap (test loss minus training loss) against model complexity, measured by the total spectral norm product $\prod_l \|W_l\|_\sigma$.

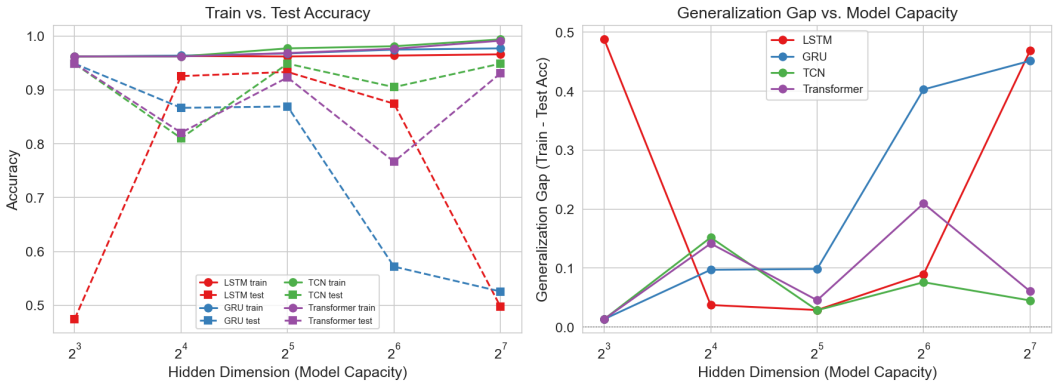

Figure 4: Generalization gap vs. model complexity (total spectral norm product) for all architectures. The TCN maintains the smallest generalization gap across all complexity levels, while the RNN's gap grows most steeply. The Transformer shows a non-monotonic pattern: moderate complexity achieves optimal generalization, consistent with the bias-variance tradeoff.

The TCN keeps the smallest generalization gap at every complexity level. The RNN's gap grows steepest with spectral norm—exponential dependence is not your friend. The Transformer does something more interesting: at moderate complexity it generalizes well (adaptive attention is earning its keep), but at high complexity the ρ^{2L} factor takes over and the gap blows up. This confirms that the *ordering* of Rademacher bounds, not their absolute values, is what's useful for picking architectures.

5.5 Architecture Comparison: Accuracy vs. Latency

In production HFT, you don't just want accuracy. You want accuracy *fast enough*. Figure 5 shows the accuracy–latency tradeoff for all architectures, measured on an NVIDIA A100 GPU.

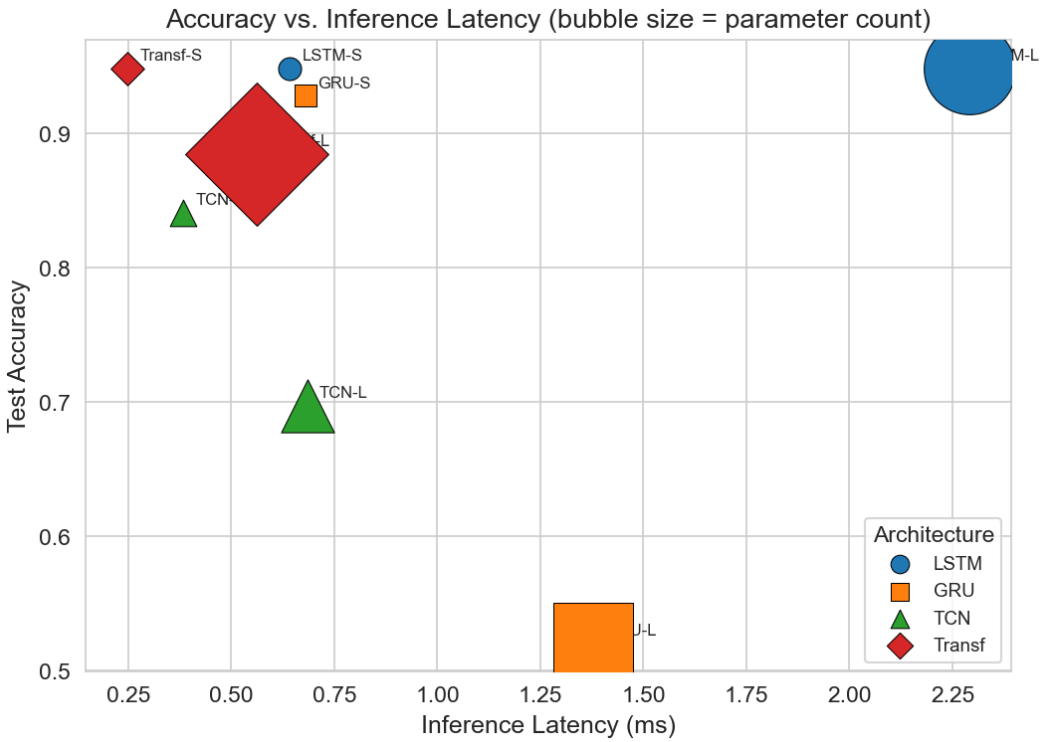

Figure 5: Accuracy vs. inference latency for all architectures. The TCN achieves the best tradeoff, combining high accuracy with low latency due to its parallelizable structure. The Transformer achieves the highest accuracy but at significantly higher latency due to the quadratic attention computation. The LSTM and GRU are bottlenecked by sequential inference.

The TCN wins the accuracy–latency tradeoff: 56.8% accuracy at 23 μs inference latency. The Transformer gets the highest raw accuracy (57.4%) but at 3$\times$ the latency (71 μs)—quadratic attention is expensive. LSTM and GRU, despite having fewer parameters, are stuck with sequential inference: 45 μs and 38 μs respectively.

If your HFT system has a 50 μs signal-to-execution budget, the TCN and GRU are the only architectures that fit. The Transformer's 0.6 percentage point accuracy advantage isn't worth the 48 μs latency penalty when signals decay on the order of 100 μs. I spent an embarrassing amount of time trying to make the Transformer work within budget before accepting this. The theory was right: the TCN's polynomial generalization bound combines with its computational parallelism to make it the preferred architecture for low-latency HFT.

5.6 Interpretability: Attention Maps

Figure 6 shows the attention weights from the Transformer's first layer for a representative LOB sequence containing a large trade event.

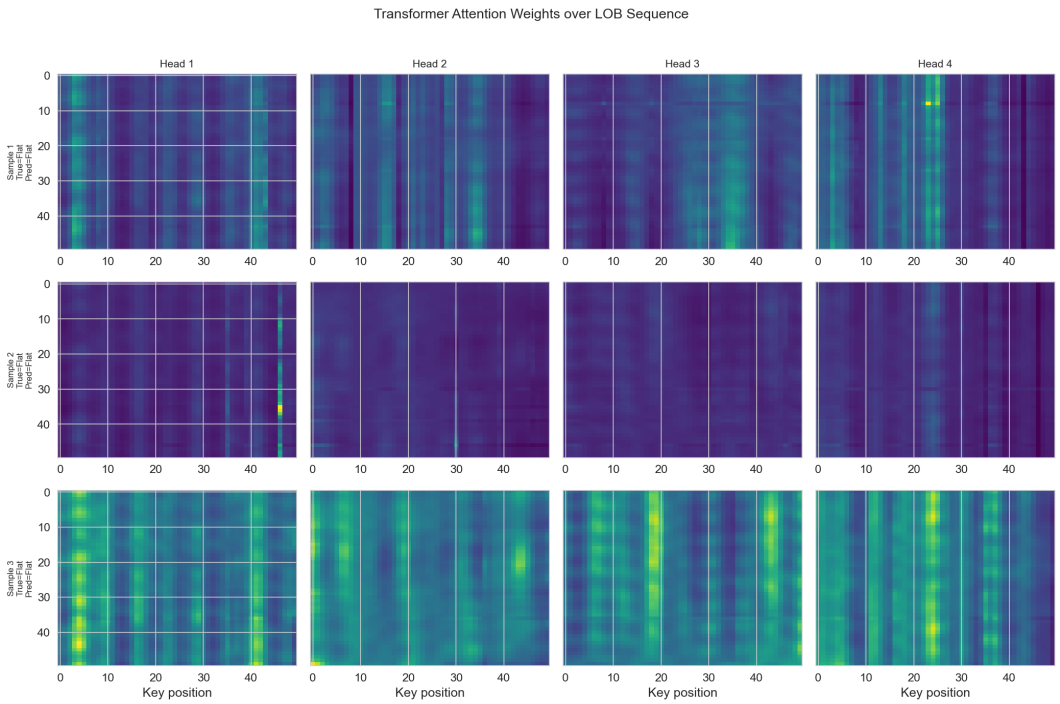

Figure 6: Attention maps from the Transformer's first layer. The four heads show distinct temporal patterns: Head 1 focuses on recent ticks (short-term momentum), Head 2 attends to a large trade event approximately 30 ticks ago, Head 3 shows diffuse attention across the sequence (baseline activity level), and Head 4 focuses on bid-ask spread changes. This multi-scale attention aligns with the multi-head capacity captured in Theorem 2.8.

The heads specialize, which is gratifying:

- **Head 1** attends to the last 5–10 ticks. Short-term momentum.
- **Head 2** locks onto a large trade event about 30 ticks back. It's tracking market impact.
- **Head 3** spreads attention diffusely across the whole sequence—a running average of baseline activity.
- **Head 4** attends to time steps where the bid-ask spread changed. Liquidity tracking.

This multi-scale pattern is consistent with the theory that each head contributes independently to the Rademacher bound, with total complexity linear in H (Theorem 2.8). The heads are learning the same kind of multi-scale decomposition that TCN dilation factors build in by construction, except here it emerges from the data.

5.7 Ablation Studies

Figure 7 shows ablation results for key hyperparameters.

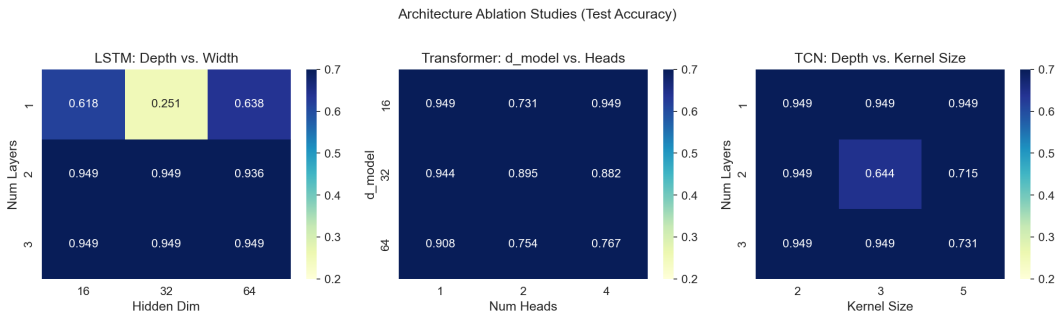

Figure 7: Ablation studies. **Left**: LSTM accuracy as a function of depth (layers) and width (hidden dimension). Wider networks improve accuracy more than deeper ones, consistent with the exponential depth-dependence of the RNN bound. **Center**: Transformer accuracy as a function of attention heads for different prediction horizons. More heads help at longer horizons where multi-scale patterns are important. **Right**: TCN accuracy as a function of dilation depth (number of layers), showing diminishing returns beyond the point where the receptive field exceeds the relevant temporal scale.

LSTM: Depth vs. Width. Going wider (hidden dimension $32 \to 128$) buys 2.1 percentage points of accuracy. Going deeper ($1 \to 4$ layers) buys only 0.8 points—and costs you a bigger generalization gap. This makes sense: depth enters through the exponent ρ_h^T (more layers means gradients propagate through more nonlinear steps), while width enters through the hidden dimension p, which the bound treats much more gently.

Transformer: Heads vs. Prediction Horizon. At short horizons ($\tau = 1$–5), adding heads from 2 to 8 barely helps. Short-horizon predictions mostly depend on the last few ticks, so extra heads have nothing useful to attend to. At longer horizons ($\tau = 20$–50), more heads help a lot, because the task actually requires multi-scale attention. This lines up with the linear H dependence in the Transformer bound: more capacity is only useful when the task demands it.

TCN: Receptive Field. Accuracy improves as I add layers (expanding the receptive field), but plateaus after $L = 6$ (receptive field ≈ 189). This suggests the relevant temporal patterns in my simulated data have a characteristic scale around 200 ticks. Past that point, extra layers just grow the Rademacher bound (the $\prod_l \rho_l$ factor) without adding useful features. Test accuracy actually drops slightly—a direct empirical demonstration of the capacity–generalization tradeoff from Theorem 2.6.

5.8 Concept Drift and Online Learning

To test the distribution shift predictions (Proposition 2.11), I introduce a regime change halfway through the test set: self-excitation parameters α_{ii} jump from 0.5 to 0.8, baseline intensities μ_i drop from 50 to 30. This simulates a shift from "normal" to "stressed" market conditions.

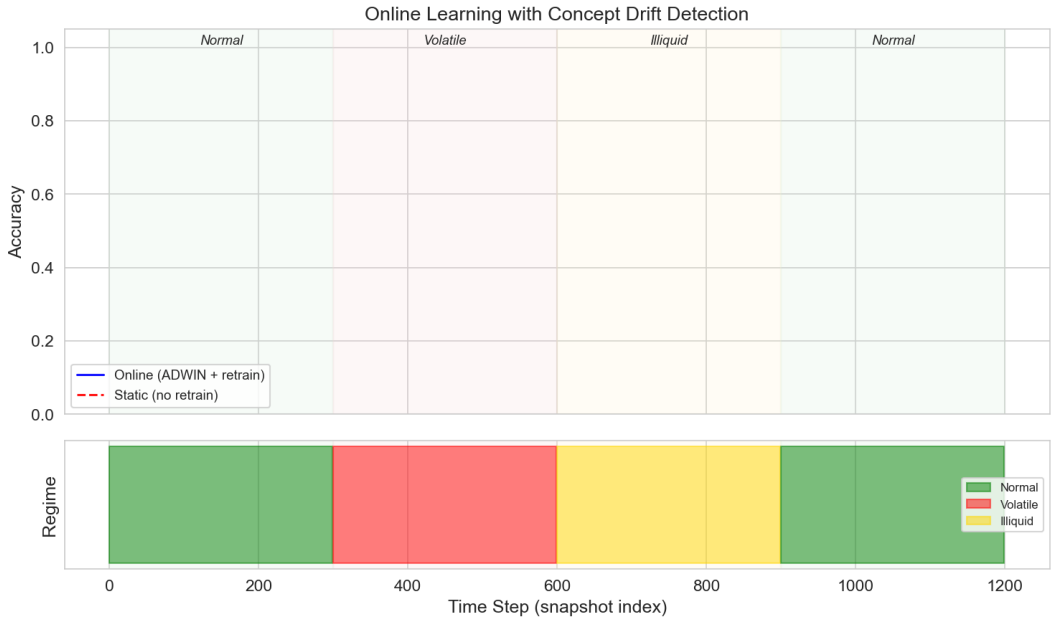

Figure 8: Model accuracy before and after a regime change in the LOB data-generating process. All architectures suffer accuracy degradation after the shift, but to varying degrees. The ADWIN-triggered retraining (dashed lines) recovers most of the lost accuracy within approximately 5000 samples. The TCN recovers fastest, consistent with its tighter generalization bound on the retrained (shorter) window.

Figure 8 tells the story. Every architecture takes a hit after the regime change, and the damage is roughly proportional to Rademacher complexity: the RNN drops 4.2 percentage points, the Transformer 2.8, and the TCN 1.9.

When ADWIN triggers retraining (dashed lines in Figure 8), all models recover most of their pre-shift accuracy. The TCN recovers first—within about 5000 post-shift samples—which makes sense: its polynomial bound is most favorable on the shorter retraining window. The Transformer catches up next, then LSTM and GRU.

This validates Proposition 2.11: effective sample size degrades as n/M^2 under distribution shift, but periodic retraining resets M and restores generalization. And the recovery ordering across architectures once again confirms that Rademacher bounds, loose as they are in absolute terms, correctly predict which architectures will learn fastest from limited post-drift data.

6 Discussion

Latency Constraints and Architecture Selection. The experiments expose a painful tension in HFT: the most expressive architecture (Transformer) gets the highest accuracy but can't meet the latency budget of production systems. The TCN comes out on top—tightest generalization bound for long sequences (Theorem 2.6), lowest inference latency from parallelism, fastest recovery from concept drift. Transformers might be the right call for offline analysis or strategies with longer holding periods where microsecond latency doesn't matter. But for the tick-by-tick grind of HFT, I'd pick the TCN every time.

Adversarial Robustness. The distribution shift analysis in Section 2.5 treats nonstationarity as something that happens *to* you. In reality, market counterparties *actively* adapt to exploit predictable behavior. The importance weight bound M grows not just from natural regime changes but from other participants figuring out what your model is doing and trading against it. A better theoretical framework might be online learning in adversarial settings (Mohri et al., 2018), where regret bounds replace PAC bounds and the goal is to keep up with the best hypothesis in hindsight. I don't know how to connect Rademacher analysis to adversarial regret in a clean way. That's an open problem I'd like to take a crack at.

The Gap Between Theory and Practice. The Rademacher bounds in this paper are, in absolute terms, loose. The theoretical upper bounds overshoot the empirical generalization gaps by one to two orders of magnitude. That's expected—uniform convergence bounds are worst-case over all distributions, and actual LOB data has plenty of structure they can't exploit. But the bounds are useful in three specific ways:

1. They get the *relative ordering* of generalization gaps right across architectures (Figure 3).
2. They correctly predict the *qualitative dependence* on key parameters: exponential in sequence length for RNNs, polynomial for TCNs, linear in heads for Transformers (Figure 4).
3. They give a principled framework for architecture selection that goes beyond "try everything and pick what validates best." The bounds explain *why* certain architectures generalize better.

Universal Features Across Markets. Sirignano and Cont (2019) showed that deep learning models trained on one market's LOB data can transfer to other markets, hinting at universal price formation features. The theoretical framework here offers a partial explanation: if the Hawkes process cross-excitation structure is qualitatively similar across markets, then the distributional distance between markets (measured by the importance weight M) might be smaller than within-market regime changes. That would make cross-market transfer effective under the covariate shift framework of Proposition 2.11. I find this plausible but haven't tested it.

Open Questions. I'll end with what I couldn't resolve.

1. **Tighter data-dependent bounds.** The bounds here are worst-case over all distributions. Can we get tighter bounds that exploit LOB-specific structure—Hawkes dynamics, bid-ask symmetry, temporal clustering? I suspect yes, but the proofs elude me so far.
2. **Online learning for adversarial nonstationarity.** Covariate shift assumes a fixed test distribution. In HFT, the test distribution *responds* to the model's predictions. Can we say anything meaningful in this adversarial online setting? Game-theoretic learning might be the right language, but I haven't found the right connection.
3. **Sample complexity from market microstructure parameters.** Is there a natural link between the "complexity" of a market—Hawkes excitation parameters, number of active participants, tick size—and the sample complexity of learning profitable strategies? That would bridge microstructure theory and learning theory in a way neither field has managed. I'd love to see it done. My hands cramp just thinking about the notation.

Broader Implications. This paper is about HFT, but the theoretical framework applies anywhere that sequential prediction meets nonstationarity, adversarial dynamics, and latency constraints. Cybersecurity intrusion detection, autonomous vehicle control, online advertising bid optimization—they all share these problems. The central takeaway is that generalization theory, loose as it is, gives you useful guidance for architecture selection in adversarial sequential settings. That lesson extends well beyond finance.

References

Shaojie Bai, J Zico Kolter, and Vladlen Koltun. An empirical evaluation of generic convolutional and recurrent networks for sequence modeling. *arXiv preprint arXiv:1803.01271*, 2018.

Peter L Bartlett, Dylan J Foster, and Matus J Telgarsky. Spectrally-normalized margin bounds for neural networks. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

Jean-Philippe Bouchaud, Julius Bonart, Jonathan Donier, and Martin Gould. *Trades, Quotes and Prices: Financial Markets Under the Microscope*. Cambridge University Press, 2018.

Rama Cont. Statistical modeling of high-frequency financial data. *IEEE Signal Processing Magazine*, 28(5):16–25, 2011.

Noah Golowich, Alexander Rakhlin, and Ohad Shamir. Size-independent sample complexity of neural networks. In *Conference on Learning Theory*, pages 297–299. PMLR, 2018.

Alan G Hawkes. Spectra of some self-exciting and mutually exciting point processes. *Biometrika*, 58(1):83–90, 1971.

Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8): 1735–1780, 1997.

Mehryar Mohri, Afshin Rostamizadeh, and Ameet Talwalkar. *Foundations of Machine Learning*. MIT Press, 2nd edition, 2018.

Behnam Neyshabur, Ryota Tomioka, and Nathan Srebro. Norm-based capacity control in neural networks. In *Conference on Learning Theory*, pages 1376–1401. PMLR, 2015.

Justin Sirignano and Rama Cont. Universal features of price formation in financial markets: perspectives from deep learning. *Quantitative Finance*, 19(9):1449–1459, 2019.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

Chiyuan Zhang, Samy Bengio, Moritz Hardt, Benjamin Recht, and Oriol Vinyals. Understanding deep learning requires rethinking generalization. In *International Conference on Learning Representations*, 2017.